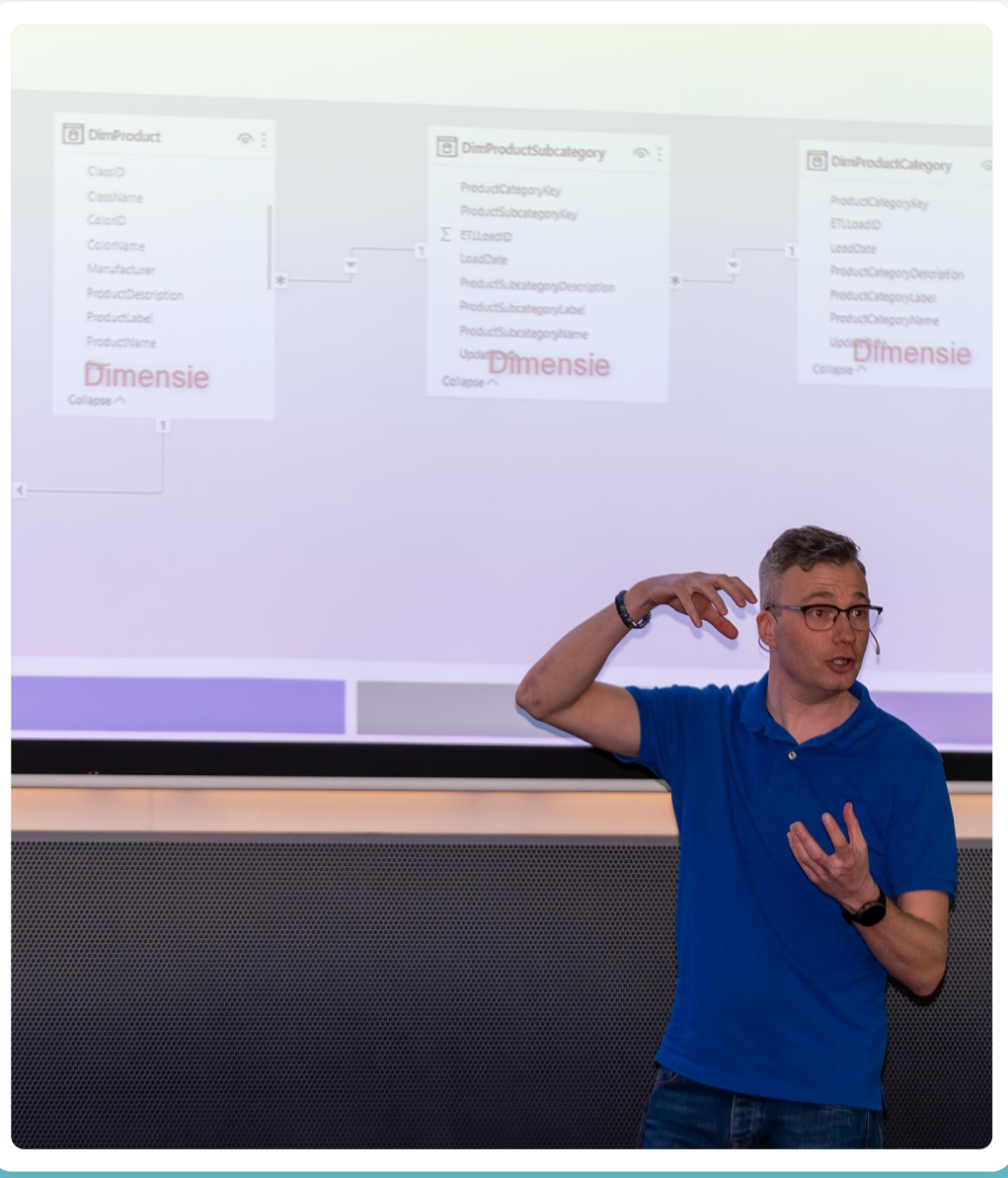
dutchBIgeek

Practical Data Modeling in Power BI: Beyond the Star Schema

Four patterns and their cost in the engine

10-10-2026



Thank you sponsors!



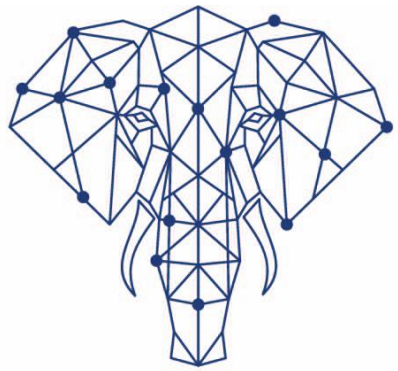
























Peter van den Bos

Data Modeling

Data Modeling & DAX Specialist
Freelance Trainer & Speaker
Founder @ DutchBIGeek

dutchbigeek.nl
linkedin.com/in/petervdbos
github.com/PetervandenBos
sessionize.com/peter-van-den-bos
youtube.com/@DutchBIgeek
peter@dutchbigeek.nl



🧠 **Geek by design**

⚡ *Because performance isn't optional*

Four Problems Beyond the Star Schema

Data Modeling

- Grain: the decision that shapes everything
- Fact type: transaction, snapshot, accumulating
- History: Slowly Changing Dimensions (SCD2)
- Many-to-many: bridge, or an alternative

Our Roadmap

Data Modeling

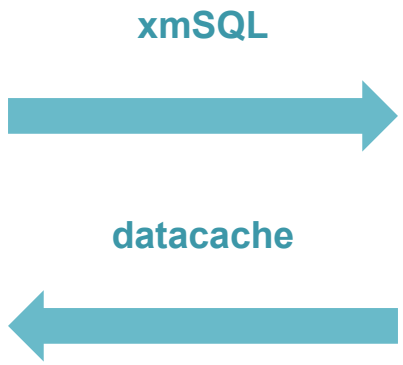
Four problems, one continuous model: each with a price.

- **Grain: the decision that shapes everything**
- **Fact type: transaction, snapshot, accumulating**
- **History: Slowly Changing Dimensions (SCD2)**
- **Many-to-many: bridge, or an alternative?**

Two engines, one query

Formula Engine

Understands all of DAX
One thread
Works on the datacache
No cache



Storage Engine

VertiPaq: reads compressed columns
Many threads
SUM, COUNT, MIN, MAX, + − ×
Results are cached

CallbackDataID: the SE asks the FE for help, for every row. For example DATEDIFF or DIVIDE. This is slow, and it is not cached.

How I read Server Timings

Data Modeling

1

SE queries

How many trips to VertiPaq?

2

FE vs SE

Where does the time go?

3

CallbackDataID

Does the SE need help?

Par. = SE CPU ÷ SE duration: how many threads did the work

Every run: Clear Cache, then Run

Four questions, four times

Data Modeling

1

Pattern

the short version, you know it

2

Silent wrong number

wrong, but no error

3

Fix

what I change in the model or measure

4

Engine bill

what it costs, live in DAX Studio

You know these patterns. Today we look at what they cost.

What Goes Wrong When Granularity Is Wrong?

Data Modeling

When grains don't align

- Different detail levels compared directly
- Totals look correct, but are conceptually wrong

Example: monthly target vs daily sales

- Two grains: target = month, sales = day
- Repeating the target per day inflates the total

This is exactly Demo 1. Up next.

Demo Time

Data Modeling

Grain mismatch, live. No safety net.

Demo 1 recap: Grain

Data Modeling

Pattern

Sales per day, quota per month. Two different grains.

Silent wrong number

Allocation looks right on every day. The total is nonsense.

Fix

Report on the grain both facts share. Or allocate, and tell the business it's an assumption.

Engine bill

Allocation in DAX: more work for the formula engine. Precompute: more rows to scan and more memory.

Granularity Drives Fact Type

Transaction Fact

- One row per business event
- Detailed, event-driven data

Snapshot Fact

- One row per entity per time period
- Captures status at a specific moment

Accumulating Snapshot Fact

- One row per process instance
- Tracks milestones within a lifecycle

Demo Time

Data Modeling

Snapshot facts: periodic & accumulating, live.

Demo 2 recap: Snapshots

Data Modeling

Pattern

Periodic snapshot: stock per day.
Accumulating snapshot: one row per order, three dates.

Silent wrong number

SUM of stock over time. The total adds up the stock of every day.

Fix

Closing balance over time. Store durations as a column.

Engine bill

The closing balance reads the fact twice. DATEDIFF per row gives a CallbackDataID.

Demo 2.2 recap: Column or measure

Data Modeling

Pattern

Order date and ship date in one row. Days to ship: a column, or DATEDIFF in a measure?

The trap

Both give the same answer. It looks fine, so nobody checks the cost.

Fix

Store DaysToShip as a column in the fact table.

Engine bill

Column: fast, but about 100 MB of memory. DATEDIFF per row: no memory, but a CallbackDataID on every query.

When Granularity and Fact Types Aren’t Enough

Data Modeling

Even with the correct grain and fact type…

- The model is right, and results can still be wrong

Why? Dimensions change over time

- History is lost
- Facts get evaluated against today's dimension values

This is what SCD2 solves. Up next.

Enter SCD Type 2

Data Modeling

How it works

- New row per change, each with its own SurrogateKey
- Facts link to the correct historical version

Typical columns

- SurrogateKey · BusinessKey · ValidFrom / ValidTo · IsCurrent

Why it matters

- “Employee was in Department A at the time of the sale”

Demo Time

Data Modeling

SCD Type 2: preserve history, and the trap, live.

Demo 3 recap: History (SCD2)

Data Modeling

Pattern

A new row and a new surrogate key for every change. The fact joins on the version.

Silent wrong number

Filter on IsCurrent and history moves to today's department. The total stays the same.

Fix

A plain SUM over the surrogate key.

Engine bill

The correct measure is also the cheapest one. The wrong one does more work.

Why M2M Breaks Your Model

Filter propagation is hard to control

- No longer strictly many-to-one: result depends on context

Rows don’t add up to the total

- One fact → many dimension rows; every row is right, their sum isn’t

Grain mismatch is often the root cause

- Budget vs sales: the relationship compensates for a modeling issue

Demo Time

Many-to-many via a bridge, live.

Demo 4 recap: Many-to-many

Data Modeling

Pattern

One sale, more employees. A bridge on the surrogate key, single direction.

Silent wrong number

The rows add up to more than the total. That's correct, but people think it's a bug.

Fix

CROSSFILTER in the measure. Only allocate when the business asks for it.

Engine bill

DIVIDE per bridge row gives a CallbackDataID. Bidirectional is not faster: the cost is in the keys.

Four demos, side by side

Data Modeling

	What goes wrong	What it costs
Grain	Total is nonsense after allocation	Work moves to the FE or to the scan
Snapshots	Stock summed over time	Two passes, DATEDIFF callback
History	IsCurrent moves history, same total	Correct measure is the cheapest
Many-to-many	Rows add up to more than the total	DIVIDE callback, cost is in the keys

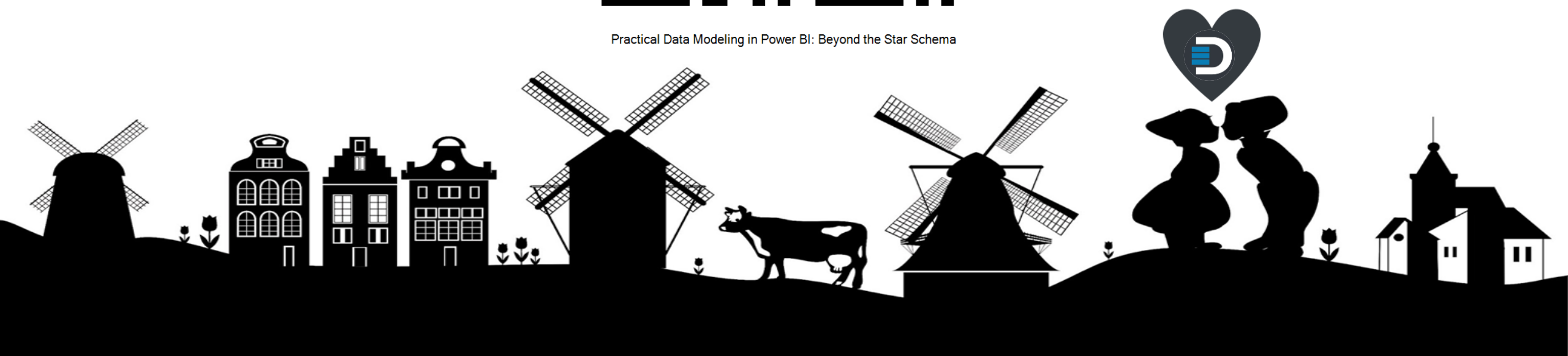
Takeaways

Every choice has a price. The engine shows you what it is.

- Always check the total row
- Look for CallbackDataID in Server Timings
- The cost doesn't go away, it moves
- Fast is not the same as correct

Rate This session

Practical Data Modeling in Power BI: Beyond the Star Schema



dutchBIgeek

If you have questions or insights, please contact me !



Peter van den Bos

Business Intelligence Consultant

peter@dutchbigeek.nl

+31 6 13760795

https://www.linkedin.com/in/petervdbos/

dutchbigeek.nl